CSC215 Lab6 – C Pointer and GDB

Total: 100pts

@Copyright Dr. Xiang Fu. All Rights Reserved.

What to turn in: a zip file containing the following: (1) CSC215Lab6.docx (2) Problem1.cc

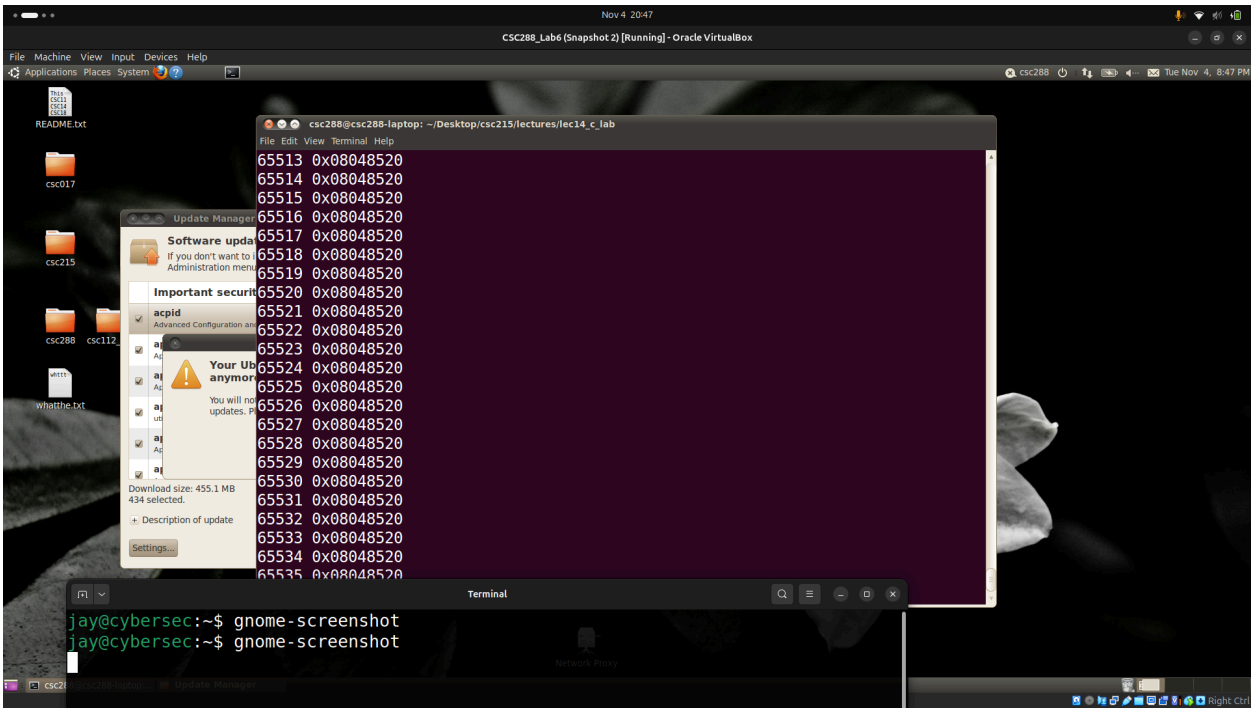
Note: the files needed for this lab are located in **~/Desktop/csc215/lectures/lec14_c_lab** in the **Ubuntu** image.

1. Printf [30pt]

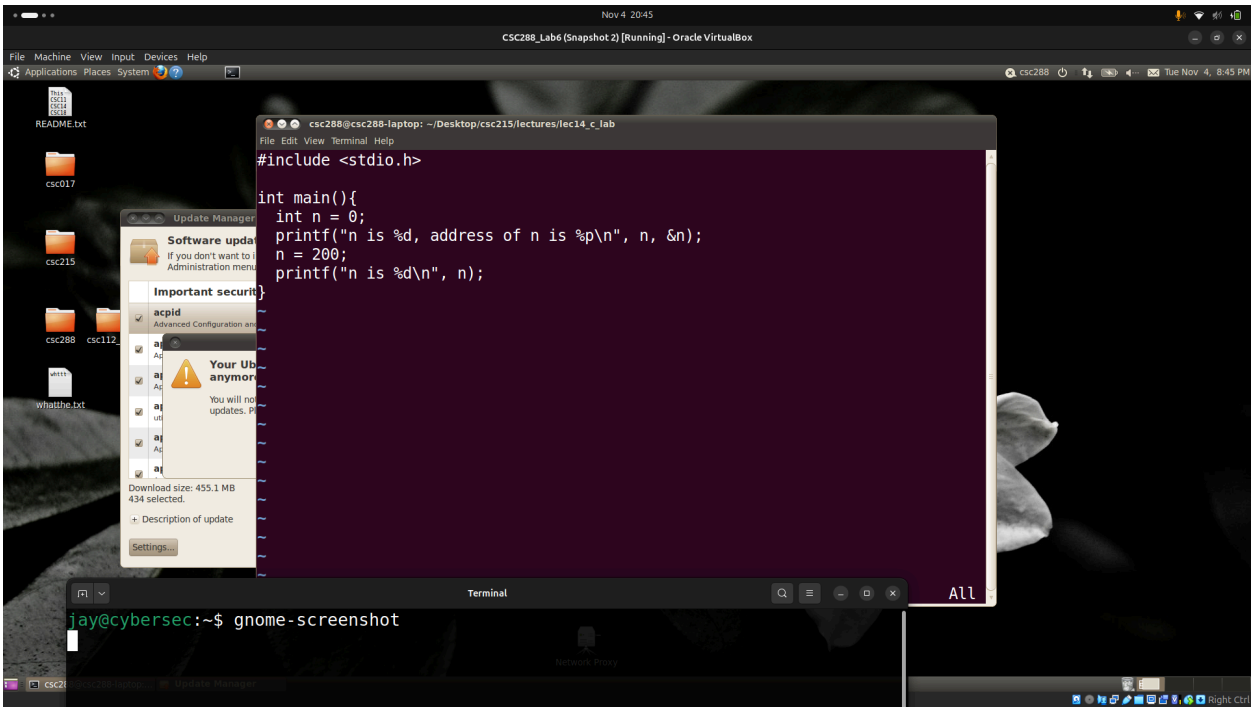
[1. Submit as **Problem1.cc** 10pt, 5 minutes] Write a simple C program which prints out integers from -65536 to 65535 in hexadecimal format. Each hex number should have 8 digits, a sample output is listed below:+

0 0x00000000
1 0x00000001
2 0x00000002

…



[2. 20pt, 5 minutes] Finish the setvar.cc (one line missing) so that at the end of the program, the value of "n" is 200.



2. C Language Pointers [30pt]

[1. 15pt, 8 minutes] Try answering the following questions without running the program below:

```
#include <stdio.h>

int main(){

  int a = 1;

  int b = 2;

  int *p = &a;

  int **q = &p;

  printf("a is located at %p\n", &a);

  printf("b is located at %p\n", &b);

}


//the output of the above program is

a is located at 0xbfcfc04c


b is located at 0xbfcfc048
```

[a] What is the value of &p? _____0xbfcfc044_____

[b] What is the value of &q? _____0xbfcfc040_____

[c] What is the value of q? _____0xbfcfc044_____

[2. 15pt, 8 minutes] Complete the above program (pointer1.cc) so that you could print out the values of &p, &q, and q. Now answer the questions again. You can use GDB to find out the answer.

a _____1_____ b _____2_____ &p _____0xbffff3c4_____

&q _____0xbffff3x0_____ q _____0xbffff3c4_____

3. GDB Practice [40pt]

Use GDB to trace into **bubble_sort.cc** and answer the following questions without modifying the source code.

[1. 10 pt, 5 minutes] what is the value of `i` and `j` when the program reaches line **8** and the value of `arr[4]` is 5?

_____1,3_____

[2. 15 pt, 8 minutes] re-run the program and stop at line 20 (before the call of sort). Find out the following using GDB (**all in hex**):

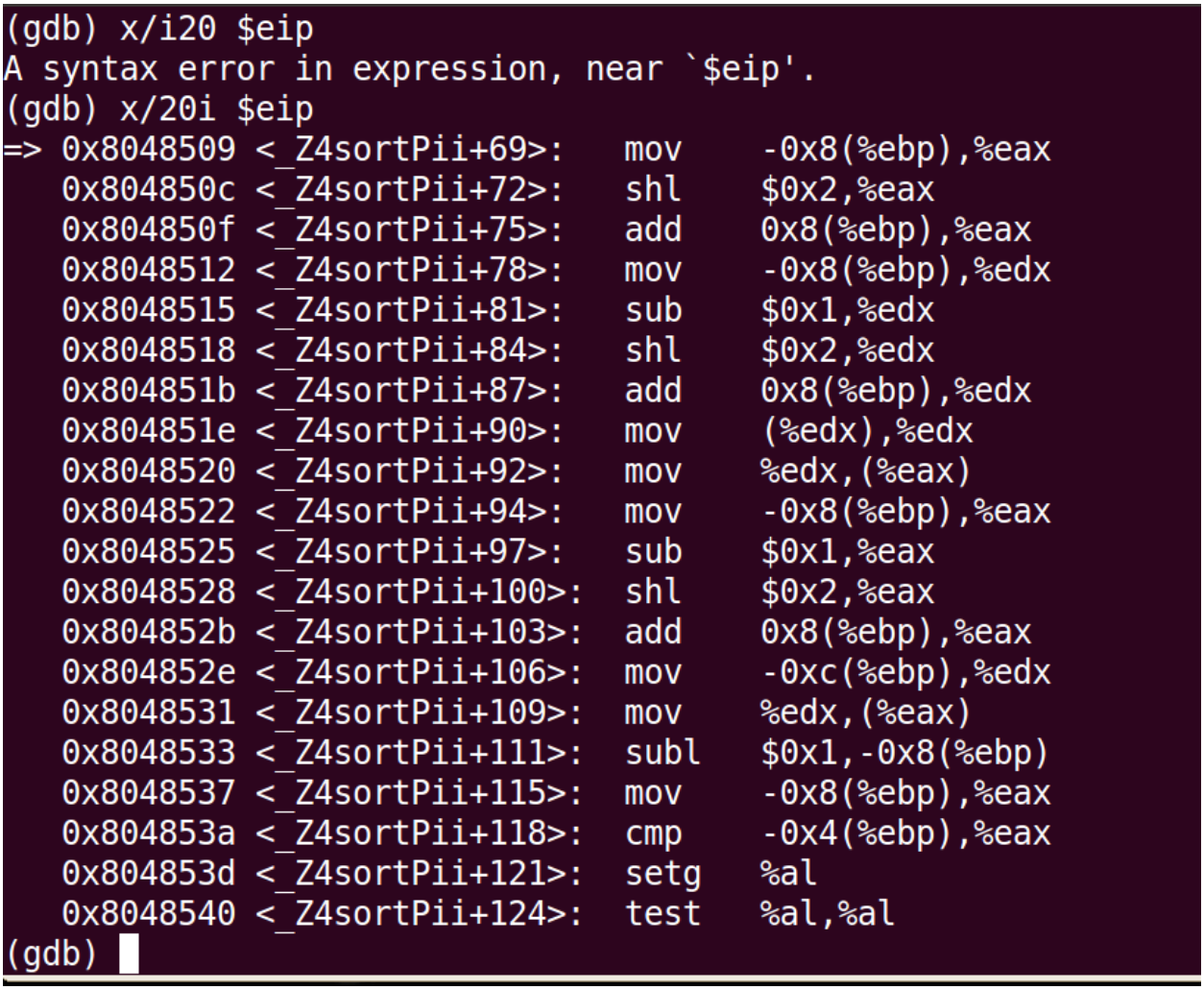
a _____1_____ b _____2_____ c _____3_____ d _____4_____

&a ____0xbffff3bc_____ &b _____0xbffff3b8_____ & c_____0xbffff3b4_____ &d _____0xbffff3b0_____

beginning address of arr _____0xbffff398_____

value of the EIP register _____0x80485ac_____

[3. 15pt, 8 minutes] List the assembly instructions that correspond to line 8 "`arr[j] = arr[j-1]`".


(gdb) x/i20 $eip
A syntax error in expression, near `$eip'.
(gdb) x/20i $eip
=> 0x8048509 <_Z4sortPii+69>:   mov   -0x8(%ebp),%eax
   0x804850c <_Z4sortPii+72>:   shl $0x2,%eax
   0x804850f <_Z4sortPii+75>:   add   0x8(%ebp),%eax
   0x8048512 <_Z4sortPii+78>:   mov   -0x8(%ebp),%edx
   0x8048515 <_Z4sortPii+81>:   sub $0x1,%edx
   0x8048518 <_Z4sortPii+84>:   shl $0x2,%edx
   0x804851b <_Z4sortPii+87>:   add   0x8(%ebp),%edx
   0x804851e <_Z4sortPii+90>:   mov   (%edx),%edx
   0x8048520 <_Z4sortPii+92>:   mov   %edx,(%eax)
   0x8048522 <_Z4sortPii+94>:   mov   -0x8(%ebp),%eax
   0x8048525 <_Z4sortPii+97>:   sub $0x1,%eax
   0x8048528 <_Z4sortPii+100>: shl $0x2,%eax
   0x804852b <_Z4sortPii+103>: add   0x8(%ebp),%eax
   0x804852e <_Z4sortPii+106>: mov   -0xc(%ebp),%edx
   0x8048531 <_Z4sortPii+109>: mov   %edx,(%eax)
   0x8048533 <_Z4sortPii+111>: subl $0x1,-0x8(%ebp)
   0x8048537 <_Z4sortPii+115>: mov   -0x8(%ebp),%eax
   0x804853a <_Z4sortPii+118>: cmp   -0x4(%ebp),%eax
   0x804853d <_Z4sortPii+121>: setg   %al
   0x8048540 <_Z4sortPii+124>: test   %al,%al
(gdb)